

توضیحات

رابط کاربری گرافیکی (GUI) یکی از اجزای مهم در بسیاری از سامانه‌های نهفته است و امکان نمایش اطلاعات و تعامل کاربر با دستگاه را فراهم می‌کند. در این پروژه، هدف طراحی و پیاده‌سازی یک رابط کاربری گرافیکی برای یک دستگاه پایش علائم حیاتی و اجرای آن بر روی یک برد نهفته واقعی است.

ابزار مورد نیاز

- برد STM32F746G-DISCOVERY
- فریم‌ورک TouchGFX یا LVGL
- مبدل USB به سریال

شرح پروژه

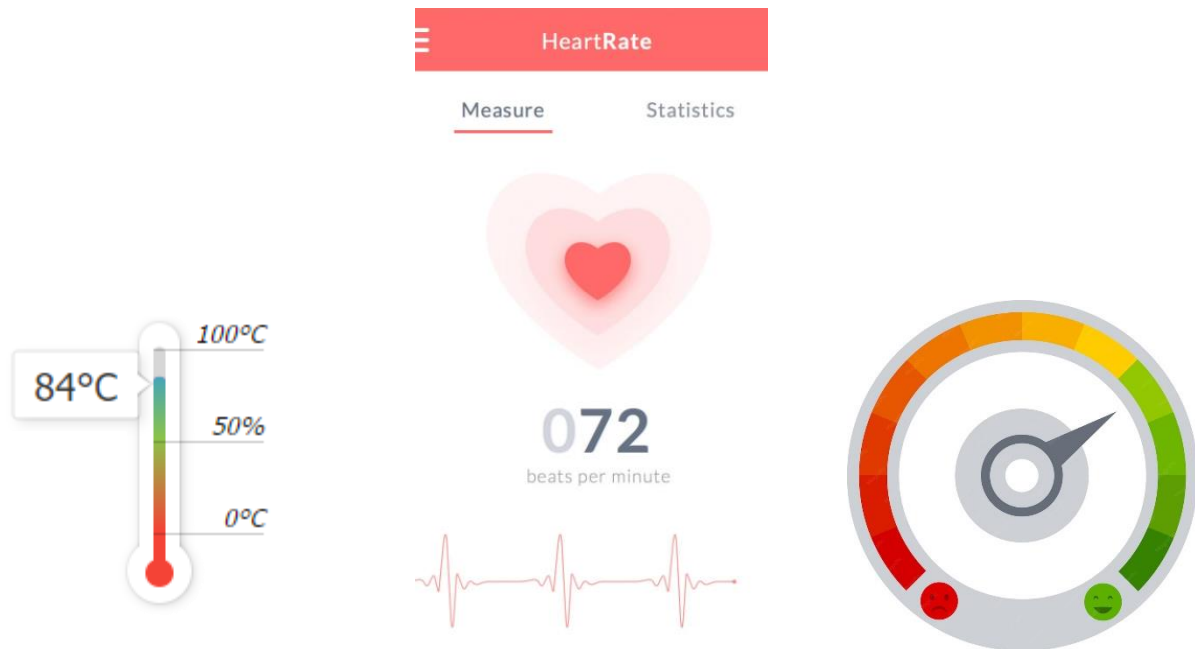
در این پروژه یک دستگاه پایش علائم حیاتی با استفاده از برد STM32F746G-

DISCOVERY طراحی و پیاده‌سازی می‌شود. تمرکز اصلی پروژه بر طراحی و

پیاده‌سازی یک رابط کاربری گرافیکی مناسب برای نمایش اطلاعات بیمار و تعامل با سامانه است. برای طراحی رابط کاربری از یکی از فریم‌ورک‌های TouchGFX یا LVGL می‌توان استفاده کرد و توسعه نرم‌افزار برد نیز می‌تواند با استفاده از STM32CubeIDE، ترکیب Keil و STM32CubeMX یا در صورت استفاده از LVGL، با استفاده از PlatformIO انجام شود.

در مرحله اول، اطلاعات مربوط به علائم حیاتی از طریق رابط UART از یک رایانه به برد ارسال می‌شود. برای این منظور، باید برنامه‌ای روی رایانه طراحی کنید که بتواند اطلاعات علائم حیاتی را تولید و از طریق UART ارسال کند. این برنامه باید اطلاعات مربوط به حداقل ۴ پارامتر **ضربان قلب (Heart Rate)**، **میزان اکسیژن خون (SpO₂)**، **دمای بدن (Body Temperature)** و **سیگنال ECG** را به صورت تصادفی اما معنادار تولید و ارسال کند. همچنین این برنامه باید امکان شبیه‌سازی شرایط مختلف بیمار را فراهم کند؛ برای مثال، سناریوهایی مانند فرد سالم، فرد دارای تکی‌کاردی (ضربان قلب بالا)، فرد دارای برادی‌کاردی (ضربان قلب پایین)، فرد دارای سطح پایین اکسیژن خون و فرد دارای دمای غیرطبیعی در نظر گرفته شود و برنامه متناسب با هر سناریو، اطلاعات مناسب را تولید و ارسال کند. داده‌های ارسالی باید در قالب JSON باشند و محدودیتی در انتخاب زبان برنامه‌نویسی برای پیاده‌سازی برنامه رایانه وجود ندارد.

در ادامه، برد STM32 باید داده‌های دریافتی را پردازش کرده را در رابط کاربری نمایش دهد. نحوه نمایش هر پارامتر باید متناسب با ماهیت آن طراحی شود؛ به‌گونه‌ای که ضربان قلب همراه با یک انیمیشن ضربان‌دار (مانند آیکون قلب متحرک با تغییر اندازه متناسب با ریتم ضربان) نمایش داده شود، SpO₂ به‌صورت عددی به همراه یک نشانگر گرافیکی مانند گیج دایره‌ای یا نوار سطح اکسیژن نمایش داده شود، دمای بدن به‌صورت عددی همراه با یک المان گرافیکی ترمومتر یا نوار رنگی تغییرپذیر متناسب با دما نمایش داده شود و سیگنال ECG به‌صورت یک نمودار پیوسته و Real-Time مشابه مانیتورهای پزشکی روی صفحه نمایش داده شود. نمونه‌هایی از المان‌های گرافیکی مناسب برای رابط کاربری در تصویر زیر آورده شده است.



در مرحله آخر، دستگاه باید امکان اتصال به شبکه از طریق رابط Ethernet موجود روی برد را داشته باشد. در صورتی که وضعیت غیرطبیعی در اطلاعات دریافتی از سنسورها تشخیص داده شود، دستگاه باید یک پیام هشدار برای کاربر ارسال کند. این پیام می‌تواند از طریق ایمیل یا Push Notification باشد.

بخش امتیازی: در صورتی که از سنسورهای بیومتریک واقعی برای نمایش اطلاعات استفاده شود، تا ۱۵٪ نمره امتیازی دارد.

مراحل انجام پروژه

- راه‌اندازی محیط توسعه
- طراحی رابط کاربری
- پیاده‌سازی برنامه شبیه‌ساز روی رایانه
- پیاده‌سازی ارتباط UART
- نمایش اطلاعات روی رابط کاربری
- تشخیص شرایط غیرطبیعی
- پیاده‌سازی ارتباط Ethernet و ارسال پیام هشدار
- آزمایش و ارزیابی نهایی

توضیحات

ESP32 یکی از ماژول‌های پرکاربرد و نسبتاً ارزان در سامانه‌های اینترنت اشیاء است که از قابلیت‌های ارتباطی متنوعی برخوردار است. در این پروژه، عملکرد ارتباطی ESP32 در لایه‌های مختلف شبکه مورد بررسی و مقایسه قرار می‌گیرد. هدف، اندازه‌گیری و تحلیل تعداد پیام‌های قابل ارسال در واحد زمان و بررسی تأثیر پروتکل‌ها و تنظیمات مختلف ارتباطی بر عملکرد سامانه است.



ابزار مورد نیاز

- برد ESP32
- محیط توسعه ESP-IDF

شرح پروژه

در این پروژه، عملکرد ارتباطی برد ESP32 در شرایط یکسان شبکه و با استفاده از پروتکل‌های مختلف مورد ارزیابی قرار می‌گیرد. ESP32 از طریق Wi-Fi به یک Access Point یا Router (که می‌تواند حتی Hot Spot گوشی باشد) متصل شده و با یک رایانه که نقش سرور را دارد، ارتباط برقرار می‌کند. کیفیت و شرایط ارتباط Wi-Fi در تمام آزمایش‌ها ثابت و در بهترین وضعیت ممکن در نظر گرفته می‌شود و تمرکز آزمایش بر بررسی عملکرد لایه‌های Transport و Application خواهد بود.

برای انجام آزمایش‌ها، باید یک چارچوب آزمایشی طراحی و پیاده‌سازی کنید که امکان ارسال تعداد زیادی پیام از ESP32 به سرور و ثبت نتایج را فراهم کند. در لایه Transport، پروتکل‌های TCP و UDP و در لایه Application، پروتکل‌های HTTP، HTTPS، MQTT و CoAP مورد بررسی قرار می‌گیرند. برای پروتکل‌هایی که دارای حالت‌ها یا تنظیمات مختلف هستند، این حالت‌ها نیز باید به‌صورت جداگانه ارزیابی شوند. برای مثال، در MQTT باید عملکرد QoS 0، QoS 1 و QoS 2 به‌صورت جداگانه بررسی شود. همچنین، در HTTP و HTTPS حالت‌های مختلف ارتباطی از جمله Request-Response و Long Polling، مورد مقایسه قرار گیرند.

برای تمام آزمایش‌ها، شرایط شبکه و سایر پارامترهای قابل تأثیر باید ثابت نگه داشته شوند. اندازه پیام در سه حالت ۱۰۰ بایت، ۱ کیلوبایت و ۱۰ کیلوبایت و مدت هر آزمایش در سه بازه ۱، ۱۰ و ۶۰ ثانیه در نظر گرفته شود. هر ترکیب از پروتکل، تنظیمات، اندازه پیام و مدت آزمایش باید به‌صورت جداگانه اجرا و نتایج آن ثبت شود.

معیارهای اصلی ارزیابی عبارت‌اند از:

- Message Rate: تعداد پیام‌های موفق ارسال شده در واحد زمان
- Latency: مدت زمان مورد نیاز برای انتقال یا تبادل یک پیام
- Message Loss: نسبت پیام‌های ارسال شده‌ای که با موفقیت دریافت نمی‌شوند
- Throughput: حجم داده منتقل شده در واحد زمان

نتایج تمامی آزمایش‌ها باید ثبت شده و با استفاده از جدول‌ها و نمودارهای مناسب با یکدیگر مقایسه شوند. در نهایت، لازم است تأثیر نوع پروتکل، تنظیمات آن و اندازه پیام بر عملکرد ارتباطی ESP32 تحلیل شود و دلایل تفاوت مشاهده‌شده میان نتایج مورد بررسی قرار گیرد.

تمام برنامه‌های مربوط به ESP32 باید با استفاده از ESP-IDF توسعه داده شوند.

مراحل انجام پروژه

- راه‌اندازی محیط توسعه ESP-IDF و بورد ESP32
- راه‌اندازی رایانه به‌عنوان سرور و ثبت‌کننده نتایج
- طراحی و پیاده‌سازی چارچوب آزمایش و اندازه‌گیری
- پیاده‌سازی ارتباط مبتنی بر TCP و UDP
- پیاده‌سازی پروتکل‌های HTTP، HTTPS، MQTT و CoAP
- اجرای آزمایش‌ها و اندازه‌گیری Message Rate، Latency، Message Loss و Throughput
- ثبت و تحلیل نتایج آزمایش‌ها
- مقایسه عملکرد پروتکل‌ها و تنظیمات مختلف

توضیحات

ثابت افزار^۱ نرم افزاری سطح پایین است که مستقیماً روی سخت افزار اجرا می شود و وظیفه راه اندازی و کنترل اجرای سخت افزاری دستگاه را بر عهده دارد. ثابت افزار در سامنه های نهفته، در عمل شامل کل پشته نرم افزاری روی آن دستگاه است. امروزه ثابت افزار اکثر سامنه های عام منظوره، UEFI^۲ است که پس از روشن کردن سامنه اجرا شده و پس از انجام یک سری عملیات مانند راه اندازی سخت افزارهای جانبی، کنترل سامنه را در اختیار سیستم عامل قرار می دهد.

بخش عمده حملات علیه سامنه های نهفته طبق گزارش ها ناشی از آسیب پذیری های نرم افزاری ثابت افزار است. از آنجا که ثابت افزار مستقیماً با سخت افزار در ارتباط است، این آسیب پذیری ها می توانند تأثیرات جدی بر امنیت و عملکرد دستگاه داشته باشند؛ بنابراین، ضروری است که امنیت ثابت افزار به طور جدی مورد آزمون قرار گیرد.

فازینگ یک روش آزمون نرم افزاری ساده و خودکار برای شناسایی باگ ها است که با اجرای برنامه تحت ورودی های متنوع انجام می شود. در عمل، این روش بسیار مؤثر بوده و منجر به کشف تعداد زیادی باگ در نرم افزارهای واقعی شده است (برای نمونه بیش از ۱۸۰۰۰ باگ از مرورگر گوگل کروم توسط آن کشف شده است).

با توجه به اهمیت شناسایی باگ های ثابت افزار و همچنین اثربخشی بالای فازینگ در کشف باگ های نرم افزاری، استفاده از این روش برای ارزیابی ثابت افزار مورد توجه قرار گرفته است. با این حال، انجام آزمون فازینگ بر روی ثابت افزارها به دلیل وابستگی شدید به سخت افزار و تنوع بالای پلتفرم های سخت افزاری، با چالش های متعددی همراه است.

در سال های اخیر، پژوهش ها و ابزارهای متنوع و مؤثری برای رفع این چالش ها ارائه شده اند. در این پروژه، هدف بازپیاپی سازی و ارزیابی روش ارائه شده در مقاله SmuFuzz: Enable Deep System Management Mode Fuzzing in Fully Featured UEFI Runtime Environment است.

ابزار مورد نیاز

- چارچوب EDK II
- شبیه ساز Qemu
- کتابخانه LibAFL

^۱ Firmware

^۲ Unified Extensible Firmware Interface

شرح پروژه

روش SmuFuzz با هدف فازینگ عمیق (SMM) System Management Mode در یک محیط کامل UEFI Runtime طراحی شده است. در این پروژه، باید محیط آزمایش و اجزای اصلی روش ارائه شده در مقاله را بازسازی کرده و با استفاده از QEMU، EDK II و LibAFL، فرایند فازینگ SMM را اجرا کنید. کد، وابستگی‌های اصلی و مستندات پروژه در [مخزن رسمی آن](#) در دسترس قرار گرفته‌اند و امکان اجرای محیط آماده پروژه از طریق Docker نیز فراهم شده است.

در ابتدای پروژه، لازم است با ساختار UEFI، محیط SMM و اصول اولیه Fuzzing آشنا شوید. سپس مقاله SmuFuzz مطالعه شده و مسئله اصلی آن، یعنی محدودیت روش‌های معمول فازینگ در فراهم کردن محیط اجرایی مناسب برای ماژول‌های SMM، بررسی شود. در ادامه، پیاده‌سازی منتشر شده SmuFuzz و اجزای مورد نیاز آن شامل EDK2، LibAFL و QEMU راه‌اندازی می‌شوند. پس از اطمینان از عملکرد صحیح محیط، تعدادی ماژول SMM انتخاب شده و برای انجام عملیات Fuzzing آماده می‌شوند. هدف پروژه بازتولید کامل تمامی آزمایش‌های مقاله نیست و انجام آزمایش روی تعداد محدودی ماژول برای ارزیابی عملکرد چارچوب کافی است.

عملیات Fuzzing باید در شرایط مشخص و قابل تکرار انجام شود و نتایج حاصل از اجرای هر ماژول ثبت گردد. معیارهای اصلی ارزیابی شامل Code Coverage، Execution Rate و تعداد Crash‌های ایجاد شده است. در بررسی Code Coverage، روند افزایش پوشش برنامه در طول زمان نیز باید ثبت شده و برای ماژول‌های مختلف با یکدیگر مقایسه شود.

Crash‌های مشاهده شده باید بررسی شوند تا مشخص شود کدام موارد قابل بازتولید هستند و آیا ناشی از رفتار واقعی برنامه بوده‌اند یا از مشکلات محیط اجرا ایجاد شده‌اند. در نهایت، نتایج آزمایش‌ها تحلیل شده و عملکرد SmuFuzz از نظر توانایی پیشروی در مسیرهای اجرایی مختلف و شناسایی رفتارهای غیرعادی ماژول‌های SMM مورد ارزیابی قرار می‌گیرد.

برای ارزیابی کنترل شده چارچوب، می‌توان یک ماژول آزمایشی SMM نیز ایجاد کرد که دارای یک خطای مشخص باشد و سپس بررسی کرد که آیا SmuFuzz قادر به تولید ورودی مناسب برای رسیدن به مسیر دارای خطا است یا خیر.

در ابتدا لازم است با معماری UEFI، محیط SMM و اصول Fuzzing آشنا شوید و سپس مقاله SmuFuzz و روش پیشنهادی آن را مطالعه

مراحل انجام پروژه

- مطالعه و تحلیل معماری و روش ارائه شده در مقاله SmuFuzz
- راه‌اندازی محیط SMM/UEFI + EDK II + QEMU
- راه‌اندازی و بررسی کدهای ارائه شده در repository رسمی SmuFuzz
- بازپیاده‌سازی اجزای اصلی fuzzing با استفاده از LibAFL
- اجرای fuzzing روی یک یا چند مؤلفه/handler مربوط به SMM
- ثبت و تحلیل coverage، تعداد execution، ورودی‌های تولید شده و crash/exception‌های مشاهده شده
- بررسی نمونه‌های پیدا شده و مشخص کردن اینکه آیا رفتار غیرعادی قابل بازتولید است یا خیر
- مقایسه نتایج به دست آمده با نتایج گزارش شده در مقاله
- بررسی و انتخاب ابزارهای فازینگ منتخب

توضیحات

در این پروژه، قصد داریم یک پلتفرم یکپارچه برای آزمون فازینگ ثابت‌افزار سامانه‌های نهفته توسعه دهیم که امکان انتخاب و استفاده از چند ابزار فازینگ مختلف را در اختیار کاربر قرار دهد.

شرح پروژه

در این پروژه، یک پلتفرم یکپارچه با رابط کاربری وب پیاده‌سازی می‌شود که امکان اجرای آزمون فازینگ بر روی ثابت‌افزار دلخواه را با استفاده از ابزارهای مختلف فازینگ فراهم می‌کند. کاربر قادر خواهد بود:

- فایل باینری ثابت‌افزار مورد نظر برای آزمون را بارگذاری نماید.
- ابزار فازینگ موردنظر خود را انتخاب نماید (حداقل از میان دو ابزار از لیست زیر). معماری سیستم باید به‌صورت ماژولار باشد، به‌گونه‌ای که امکان افزودن ابزارهای فازینگ جدید با حداقل تغییر فراهم باشد.

Fuzzware, Halucinator, uEmu, Ember-IO, FirmHybridFuzzer, Hoedur, Jetset, MultiFuzz, SafireFuzz, P2IM

- تنظیمات اولیه فرآیند فازینگ شامل پارامترهای عمومی و تنظیمات اختصاصی هر ابزار فازینگ است. همچنین در صورت نیاز ابزار، ورودی‌های اولیه^۳ نیز وارد می‌شوند.
- نتایج حاصل از فازینگ را در قالب گزارش مشاهده نماید و همچنین تاریخچه اجراها قابل دسترسی باشد. گزارش خروجی شامل مواردی نظیر باگ‌های کشف‌شده، تعداد کرش‌های ثبت‌شده و پوشش کد خواهد بود. همچنین نمونه‌های آزمون^۴ که منجر به بروز کرش شده‌اند و همچنین مجموعه نهایی ورودی‌ها^۵ نیز باید قابل مشاهده و دانلود باشند.

بخش امتیازی: نتایج آزمون‌های فازینگ به‌صورت **تجمیعی** ارائه شود. در این بخش، برای هر ثابت‌افزار یکتا و هر ابزار فازینگ، تعداد دفعات اجرا و نتایج حاصل از آن‌ها خلاصه می‌گردد. این نتایج به‌صورت **گزارش‌های خلاصه یا نمودارهای مقایسه‌ای** (مانند میانگین پوشش کد یا میانگین تعداد کرش‌ها) و به‌ازای هر ثابت‌افزار یا هر ابزار فازینگ نمایش داده شوند.

توجه: در پیاده‌سازی بخش فرانت‌اند، استفاده از هر نوع تکنولوژی یا فریم‌ورکی مجاز است؛ با این حال، رابط کاربری باید از نظر طراحی در سطح مطلوبی ارائه شود. برای پیاده‌سازی بخش بک‌اند، پیشنهاد می‌شود از فریم‌ورک جنگو استفاده گردد.

³ Seed

⁴ Test case

⁵ Corpus

مراحل انجام پروژه

- مطالعه مفاهیم آزمون فازینگ
- بررسی و انتخاب ابزارهای فازینگ منتخب
- آماده‌سازی محیط اجرای ابزارهای فازینگ
- طراحی معماری پلتفرم یکپارچه
- پیاده‌سازی رابط کاربری و بخش بک‌اند
- یکپارچه‌سازی ابزارهای فازینگ با پلتفرم
- تست اجرای آزمون‌های فازینگ بر روی ثابت‌افزارهای نمونه
- مستندسازی نهایی پروژه

توضیحات

برخلاف نرم‌افزارهای معمول، در بسیاری از موارد کد منبع ثابت‌افزار در دسترس نیست و تنها فایل باینری آن در اختیار قرار دارد. در چنین شرایطی، برای بررسی عملکرد برنامه، شناسایی اجزای مورد استفاده و تحلیل رفتار دستگاه باید از روش‌های مهندسی معکوس^۶ استفاده شود.

مهندسی معکوس ثابت‌افزار شامل بررسی فایل باینری و استخراج اطلاعاتی مانند معماری پردازنده، ساختار حافظه، توابع برنامه، Peripheral‌های مورد استفاده و جریان اجرای کد است. ترکیب روش‌های تحلیل ایستا و تحلیل پویا امکان درک دقیق‌تری از رفتار ثابت‌افزار را فراهم می‌کند.

در این پروژه، یک فایل باینری ثابت‌افزار با نام `firmware.bin` مربوط به یک سامانه نهفته در اختیار شما قرار می‌گیرد. هدف پروژه این است که بدون دسترسی به کد منبع، ساختار و منطق عملکرد این ثابت‌افزار را استخراج کرده و یافته‌های خود را با اجرای آن در یک محیط شبیه‌سازی مناسب اعتبارسنجی کنید.

ابزار مورد نیاز

- Ghidra یا IDA Pro
- QEMU
- ابزارهای تحلیل باینری مانند Binwalk، strings و hexdump

شرح پروژه

در ابتدای پروژه، باید فایل باینری ثابت‌افزار ارائه‌شده را بررسی کرده و اطلاعات اولیه مربوط به ساختار آن را استخراج کنید. با استفاده از ابزارهایی مانند Binwalk، strings و تحلیل محتوای باینری، باید اطلاعات اولیه موجود در فایل را بررسی کرده و سپس معماری پردازنده هدف را شناسایی کنید.

در ادامه، فایل باینری باید در IDA Pro یا Ghidra بارگذاری شده و ساختار برنامه مورد تحلیل قرار گیرد. در این مرحله باید نقطه شروع اجرای برنامه، ساختار حافظه و بخش‌های مختلف کد و داده را شناسایی کنید. همچنین در صورت وجود ساختارهایی مانند Vector Table، اطلاعات موجود در آن باید استخراج شده و برای شناسایی Handlerها و جریان اولیه اجرای برنامه مورد استفاده قرار گیرد.

پس از شناسایی ساختار اولیه، باید کد ماشین ثابت‌افزار را تحلیل کرده و توابع اصلی برنامه را شناسایی کنید. هدف این مرحله، استخراج Control Flow و درک منطق کلی عملکرد ثابت‌افزار است. باید مشخص کنید که پس از راه‌اندازی سامانه چه بخش‌هایی از برنامه اجرا می‌شوند، توابع اصلی چه ارتباطی با یکدیگر دارند و حلقه اصلی یا رویدادهای اصلی سیستم چگونه مدیریت می‌شوند.

^۶ Reverse Engineering

بخش مهم دیگری از پروژه، شناسایی Peripheral های مورد استفاده توسط ثابت افزار است. باید دسترسی های حافظه و رجیسترهای سخت افزاری را بررسی کرده و Peripheral هایی مانند GPIO، UART، SPI و I2C را شناسایی کنید. همچنین باید مشخص شود هر Peripheral در چه بخشی از برنامه مورد استفاده قرار گرفته و چه نقشی در عملکرد کلی ثابت افزار دارد.

در صورتی که ثابت افزار با تجهیزات خارجی تبادل داده انجام می دهد، باید نحوه این ارتباط نیز بررسی شود. برای مثال، در صورت استفاده از UART یا SPI، توابع مرتبط با ارسال و دریافت داده شناسایی شده و تا حد امکان ساختار یا قالب پیام های مبادله شده استخراج شود.

پس از انجام تحلیل ایستا، باید با استفاده از QEMU یا ابزار Emulation مناسب، رفتار ثابت افزار را در زمان اجرا بررسی کنید. در این مرحله، باید نتایج به دست آمده از مهندسی معکوس با رفتار مشاهده شده در زمان اجرا مقایسه شوند. برای مثال، اگر در تحلیل ایستا استفاده از UART، GPIO یا یک بخش مشخص از حافظه شناسایی شده است، باید رفتار متناظر آن در محیط اجرا نیز بررسی و اعتبارسنجی شود.

در نهایت، باید با ترکیب نتایج تحلیل ایستا در IDA Pro یا Ghidra و نتایج تحلیل پویا در QEMU، منطق کلی عملکرد ثابت افزار را استخراج کنید. گزارش نهایی باید نشان دهد ثابت افزار چه کاری انجام می دهد، جریان اصلی اجرای آن چگونه است، از چه Peripheral هایی استفاده می کند و نتایج تحلیل با رفتار واقعی برنامه تا چه اندازه تطابق دارد.

در این پروژه موارد زیر باید بررسی و مستندسازی شود:

۱. معماری پردازنده و ساختار کلی ثابت افزار شناسایی شود.
۲. بخش های اصلی حافظه، Vector Table و نقطه شروع اجرای برنامه استخراج شود.
۳. Peripheral های مورد استفاده مانند GPIO، SPI، UART و ... شناسایی شود.
۴. توابع اصلی، جریان اجرای برنامه و منطق عملکرد ثابت افزار تحلیل شود.
۵. نحوه تبادل داده با Peripheral های خارجی استخراج شود.
۶. رفتار ثابت افزار در محیط شبیه سازی بررسی و نتایج تحلیل اعتبارسنجی شود.
۷. یافته های حاصل از تحلیل در قالب یک گزارش فنی ارائه شود.

مراحل انجام پروژه

- مطالعه ساختار ثابت افزارهای نهفته و روش های مهندسی معکوس
- تحلیل اولیه فایل باینری و شناسایی معماری هدف
- استخراج ساختار حافظه و اجزای اصلی ثابت افزار
- تحلیل کد ماشین و شناسایی Peripheral های مورد استفاده
- استخراج منطق عملکرد و جریان اجرای برنامه
- بررسی رفتار ثابت افزار در محیط شبیه سازی
- مقایسه نتایج تحلیل ایستا و اجرای ثابت افزار
- مستندسازی نتایج نهایی

شرح پروژه

در این پروژه، فرآیند ارزیابی امنیتی ثابت افزارهای واقعی با استفاده از ابزارهای فازینگ مخصوص سامانه های نهفته انجام خواهد شد. هدف پروژه، آشنایی عملی با روش های کشف آسیب پذیری در ثابت افزار و تحلیل فنی نتایج حاصل از اجرای ابزارهای فازینگ است.

در این پروژه، یکی از ابزارهای مطرح در حوزه Firmware Fuzzing از بین ابزارهای زیر انتخاب شود:

Fuzzware, Halucinator, uEmu, Ember-IO, FirmHybridFuzzer, Hoedur Jetset, MultiFuzz, SafireFuzz, P2IM

ابزار انتخاب شده را روی یک یا چند ثابت افزار واقعی اجرا کنید و Crash ها و Hang های تولید شده توسط فازر را جمع آوری و بررسی نمایید. سپس از بین نتایج به دست آمده، باید حداقل به یکی از Crash ها یا آسیب پذیری های گزارش شده در مقاله اصلی ابزار برسید و آن را با نتایج اجرای خود تطبیق دهید. مورد انتخاب شده می تواند دارای شناسه CVE یا دسته بندی CWE باشد، اما وجود این شناسه الزامی نیست.

در ادامه، جزئیات فنی مورد شناسایی شده شامل ورودی ایجاد کننده Crash، مسیر اجرای برنامه، بخش آسیب پذیر در ثابت افزار، نحوه رخداد خطا و علت اصلی ایجاد آسیب پذیری را بررسی و مستندسازی کنید.

مراحل انجام پروژه

- مطالعه مفاهیم Firmware Fuzzing و ابزار انتخاب شده
- آماده سازی محیط اجرا و ثابت افزار هدف
- اجرای ابزار فازینگ و جمع آوری Crash ها و Hang ها
- تطبیق نتایج فازینگ با موارد گزارش شده در مقاله ابزار
- تحلیل فنی مورد شناسایی شده و مستندسازی نتایج

توضیحات

در سامانه‌های اینترنت اشیاء، پس از استقرار دستگاه‌ها ممکن است برای رفع خطا، افزودن قابلیت‌های جدید یا بهبود امنیت، نیاز به به‌روزرسانی ثابت‌افزار وجود داشته باشد. OTA^۷ امکان انجام این به‌روزرسانی را از طریق شبکه و بدون دسترسی فیزیکی به دستگاه فراهم می‌کند. در این پروژه، یک سامانه مدیریت OTA برای بوردهای ESP32 طراحی و پیاده‌سازی می‌کنید که قابلیت مدیریت نسخه‌های ثابت‌افزار، به‌روزرسانی از راه دور، بررسی صحت فایل دریافتی و بازگشت به نسخه قبلی در صورت بروز خطا را فراهم می‌کند.

ابزار مورد نیاز

- برد ESP32

شرح پروژه

در این پروژه، باید یک ثابت‌افزار برای ESP32 توسعه دهید که علاوه بر منطق اصلی برنامه، دارای یک ماژول مدیریت OTA باشد. این ماژول باید بتواند اطلاعات دستگاه، از جمله شناسه و نسخه فعلی ثابت‌افزار را برای سرور ارسال کرده و وجود نسخه جدید را بررسی کند.

در سمت سرور، باید امکان مدیریت دستگاه‌ها و نسخه‌های مختلف ثابت‌افزار فراهم شود. از طریق پنل مدیریتی تحت وب باید بتوانید دستگاه‌های ثبت‌شده را مشاهده کرده و اطلاعاتی مانند شناسه دستگاه، نسخه فعلی ثابت‌افزار و وضعیت آخرین به‌روزرسانی را بررسی کنید.

کاربر از طریق پنل مدیریتی قادر خواهد بود:

- دستگاه‌های ثبت‌شده را مشاهده کرده و اطلاعاتی مانند شناسه دستگاه و نسخه فعلی ثابت‌افزار را مشاهده نماید.
 - فایل ثابت‌افزار جدید را در سامانه بارگذاری کرده و اطلاعات نسخه مانند شماره نسخه و توضیحات تغییرات را ثبت نماید.
 - عملیات به‌روزرسانی OTA را برای دستگاه موردنظر انجام دهد.
 - حالت به‌روزرسانی دستگاه را تعیین نماید؛ به‌گونه‌ای که به‌روزرسانی به‌صورت خودکار پس از انتشار نسخه جدید انجام شود یا تنها پس از تأیید مدیر از طریق پنل اجرا گردد.
 - تاریخچه نسخه‌های ثابت‌افزار و عملیات‌های به‌روزرسانی را مشاهده نماید.
 - در صورت بروز مشکل پس از به‌روزرسانی، امکان بازگرداندن دستگاه به نسخه قبلی ثابت‌افزار را فراهم نماید.
- در سمت ESP32، فرآیند بررسی وجود نسخه جدید، دریافت ثابت‌افزار، بررسی صحت فایل (با استفاده از Hash یا Checksum قبل از نصب)، نصب نسخه جدید و بازگشت به نسخه قبلی در صورت شکست عملیات پیاده‌سازی خواهد شد.

مراحل انجام پروژه

- مطالعه مفاهیم OTA، ساختار ثابت‌افزار و مکانیزم Rollback در ESP32
- طراحی معماری سامانه و ارتباط بین ESP32، سرور و پنل مدیریتی

⁷ Over-the-Air Update

- پیاده‌سازی مدیریت نسخه‌های ثابت‌افزار و فرآیند OTA Update
- پیاده‌سازی بررسی صحت ثابت‌افزار و مکانیزم بازگشت به نسخه قبلی
- پیاده‌سازی پنل مدیریتی تحت وب
- مستندسازی نهایی پروژه

توضیحات

در بسیاری از سامانه‌های اینترنت اشیا، لازم است داده‌های تجهیزات و سنسورهایی با رابط‌های ارتباطی متفاوت دریافت، پردازش و از طریق شبکه برای سرور ارسال شوند. در این پروژه، یک Gateway هوشمند مبتنی بر ESP32 طراحی و پیاده‌سازی می‌کنید که قابلیت دریافت داده از سنسورها، اعتبارسنجی و ذخیره موقت داده‌ها و ارسال آن‌ها از طریق پروتکل‌های مختلف شبکه را داشته باشد.

ابزار مورد نیاز

- برد ESP32
- سنسور با رابط سریال

شرح پروژه

در این پروژه، یک Gateway هوشمند مبتنی بر ESP32 طراحی و پیاده‌سازی می‌شود که داده‌های یک سنسور با رابط SPI، I²C یا UART را دریافت کرده، پردازش اولیه را روی آن‌ها انجام داده و از طریق شبکه به مقصد ارسال می‌کند. کلیه تنظیمات سامانه از طریق پنل مدیریتی تحت وب که روی خود ESP32 اجرا می‌شود، بدون نیاز به تغییر یا کامپایل مجدد کد، قابل پیکربندی خواهد بود. کاربر از طریق پنل مدیریتی قادر خواهد بود:

- پروتکل ورودی را از میان SPI، I²C یا UART انتخاب نماید. تنظیمات مربوط به هر پروتکل از پیش در سامانه تعریف شده و تنها انتخاب پروتکل از طریق پنل مدیریتی انجام می‌شود.
- پروتکل خروجی را از میان HTTP یا MQTT انتخاب کرده و تنظیمات مربوط به آن از جمله آدرس سرور، شماره پورت، نام کاربری، رمز عبور، Topic یا API Key را بدون نیاز به تغییر کد پیکربندی نماید.
- قوانین اعتبارسنجی داده را تعریف نماید؛ از جمله تعیین حداقل و حداکثر مقدار مجاز و فعال یا غیرفعال کردن هر قانون. همچنین در صورت نامعتبر بودن داده، نحوه برخورد با آن (مانند عدم ارسال یا ارسال اعلان از طریق HTTP یا MQTT) را مشخص نماید.
- قابلیت ذخیره‌سازی موقت^۸ را فعال یا غیرفعال کرده و پارامترهایی مانند حداکثر تعداد داده‌های قابل ذخیره یا حداکثر زمان نگهداری داده‌ها را تعیین نماید. در صورت قطع ارتباط با سرور، داده‌ها در حافظه ذخیره شده و پس از برقراری مجدد ارتباط، به ترتیب ارسال خواهند شد.

مراحل انجام پروژه

- مطالعه مفاهیم پروتکل‌های ارتباطی

^۸ Cache

- طراحی معماری سامانه
- پیاده‌سازی دریافت داده از سنسور
- پیاده‌سازی تبدیل پروتکل‌های ارتباطی
- پیاده‌سازی اعتبارسنجی داده‌ها
- پیاده‌سازی مکانیزم ذخیره‌سازی موقت داده‌ها
- پیاده‌سازی پنل مدیریتی تحت وب
- تست عملکرد سامانه
- مستندسازی نهایی پروژه

توضیحات

آسیب‌پذیری‌های ناشی از Transient Execution و به‌ویژه خانواده حملات Spectre یکی از مهم‌ترین چالش‌های امنیت سامانه‌های نرم‌افزاری و سخت‌افزاری است. ابزار KASPER که در قالب یک مقاله پژوهشی^۹ منتشر شده است، رویکردی جدید برای شناسایی این دسته از آسیب‌پذیری‌ها ارائه می‌دهد. برخلاف ابزارهای پیشین که تنها به دنبال الگوهای مشخصی از کدهای آسیب‌پذیر بودند (Static Analysis)، KASPER با بهره‌گیری از Dynamic Analysis رفتار واقعی برنامه را در زمان اجرا تحلیل کرده و مسیر انتقال داده‌های تحت کنترل مهاجم، دسترسی به اطلاعات محرمانه و امکان نشت آن‌ها از طریق کانال‌های جانبی را بررسی می‌کند. خوشبختانه هم مقاله و هم کد منبع این ابزار به‌صورت عمومی در دسترس هستند و امکان مطالعه، اجرا، توسعه و استفاده مجدد از آن‌ها وجود دارد.

ابزار مورد نیاز

- کد منبع ابزار KASPER
- سیستم‌عامل Linux
- LLVM/Clang
- QEMU
- کد منبع Mbed-TLS و Zephyr RTOS
- ابزارهای Build و Debug موردنیاز برای کامپایل و اجرای پروژه‌ها

شرح پروژه

هدف این پروژه، مطالعه دقیق معماری KASPER و استفاده از آن برای تحلیل نرم‌افزارهایی فراتر از کرنل لینوکس، به‌ویژه Zephyr RTOS و Mbed-TLS است. پس از مطالعه مقاله، باید ساختار پیاده‌سازی ابزار و سازوکار تولید گزارش‌های امنیتی را بررسی کرده و تغییرات لازم برای سازگارسازی ابزار با فضای کاربری (User Space) و سیستم‌عامل Zephyr را پیاده‌سازی کنید.

علاوه بر اجرای ابزار روی این پروژه‌ها، باید فرآیند نصب، وابستگی‌ها، تغییرات اعمال‌شده، مشکلات فنی، مسیر حل مسئله، نتایج آزمایش‌ها و تحلیل خروجی‌ها را به‌صورت کامل مستندسازی کنید.

در پایان، باید قابلیت‌ها و محدودیت‌های KASPER در تحلیل پروژه‌های جدید را ارزیابی کرده و در صورت نیاز، پیشنهادهایی برای بهبود معماری ابزار، افزایش پوشش تحلیل و توسعه آن برای سایر نرم‌افزارهای امنیتی و سامانه‌های نهفته ارائه کنید.

مراحل انجام پروژه

- مطالعه مفاهیم Transient Execution و حملات Spectre
- مطالعه مقاله KASPER و آشنایی با معماری ابزار

⁹ Johannesmeyer, Brian, Jakob Koschel, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. "Kasper: Scanning for Generalized Transient Execution Gadgets in the Linux Kernel." In NDSS, vol. 1, p. 12. 2022.

- راه‌اندازی و اجرای نسخه اصلی KASPER
- بررسی ساختار پیاده‌سازی و نحوه تولید گزارش‌های امنیتی
- بررسی تغییرات لازم برای استفاده از KASPER در فضای User Space
- اعمال تغییرات لازم و اجرای ابزار روی Mbed-TLS
- بررسی امکان سازگارسازی و اجرای KASPER روی Zephyr RTOS
- تحلیل و مقایسه نتایج به‌دست‌آمده
- بررسی قابلیت‌ها و محدودیت‌های KASPER در تحلیل نرم‌افزارهای جدید
- مستندسازی فرآیند اجرا، تغییرات، مشکلات و نتایج پروژه

توضیحات

در بسیاری از کاربردهای اینترنت اشیاء، داده‌های کاربران باید پردازش شوند، در حالی که اطلاعات خام آن‌ها نباید در اختیار سرور یا سایر اجزای میانی سیستم قرار گیرد. یکی از روش‌های مناسب برای حل این مسئله، استفاده از Fully Homomorphic Encryption (FHE) است که امکان انجام محاسبات روی داده‌های رمزنگاری شده را بدون نیاز به رمزگشایی فراهم می‌کند. در این پروژه، یک سامانه پردازش امن داده با معماری Client-Edge-Server طراحی و پیاده‌سازی می‌کنید که در آن داده‌های کاربران در تمام مراحل پردازش محرمانه باقی می‌مانند.

ابزار مورد نیاز

- کد منبع ابزار KASPER
- کد منبع Zephyr RTOS و Mbed-TLS
- QEMU
- ابزارهای Build و Debug مورد نیاز برای کامپایل و اجرای پروژه‌ها

شرح پروژه

برای ارزیابی سامانه، سناریوی یک ساختمان هوشمند با ۲۰۰ واحد در نظر گرفته می‌شود. هر واحد دارای یک Client است که هر ۱۵ دقیقه یک مقدار داده مربوط به مصرف آب تولید می‌کند. داده‌ها ابتدا به یک دستگاه Edge و سپس به Server ارسال می‌شوند. سرور باید بتواند صورت‌حساب هر واحد و همچنین شاخص‌های آماری مانند میانگین و واریانس داده‌ها را محاسبه کند، بدون آنکه به مقادیر واقعی مصرف کاربران دسترسی داشته باشد.

پروژه باید در دو نسخه مستقل پیاده‌سازی شود. نسخه نخست، نسخه Plain است که در آن هیچ رمزنگاری هم‌ریختی انجام نشده و داده‌ها به‌صورت مستقیم میان Client، Edge و Server منتقل می‌شوند. این نسخه به‌عنوان مبنا برای بررسی صحت عملکرد و مقایسه کارایی سامانه استفاده خواهد شد.

نسخه دوم بر پایه روش Transciphering پیاده‌سازی می‌شود. در این روش، داده در سمت Client ابتدا با یک الگوریتم رمزنگاری متقارن غیرهم‌ریخت (Non-FHE Symmetric Cipher) رمز می‌شود. سپس در Edge، بدون آشکار شدن داده اصلی، متن رمز به داده‌ای رمز شده تحت FHE تبدیل می‌شود. پس از آن، Server محاسبات موردنظر را مستقیماً روی داده‌های رمز شده انجام می‌دهد و تنها دارنده کلید خصوصی FHE قادر به رمزگشایی نتیجه نهایی خواهد بود.

برای پیاده‌سازی عملیات رمزنگاری هم‌ریخت باید از کتابخانه [OpenFHE](#) استفاده کنید. در این پروژه‌ی متن‌باز، با توجه به نوع عملیات، Scheme‌های BFV و BGV برای داده‌های صحیح، CKKS برای داده‌های اعشاری و در صورت نیاز FHEW برای عملیات مقایسه مورد بررسی قرار می‌گیرند. همچنین باید با عملیات اصلی OpenFHE شامل تولید کلید، رمزنگاری و رمزگشایی، جمع و ضرب هم‌ریخت، مدیریت نویز، Evaluation Key، Rotation و Serialization داده‌ها آشنا شده و از آن‌ها در پیاده‌سازی استفاده کنید.

ارتباط میان Client، Edge و Server باید از طریق TCP انجام شود. داده‌های رمز شده، کلیدهای مورد نیاز و سایر اطلاعات لازم باید به صورت مناسب Serialize شده و از طریق شبکه منتقل شوند.

برای ارزیابی عملکرد، باید نسخه Plain و نسخه مبتنی بر Transciphering را از نظر زمان پردازش، زمان انتقال داده، حجم داده منتقل شده و زمان انجام محاسبات با یکدیگر مقایسه کنید. همچنین باید تأثیر نوع Scheme و پارامترهای FHE بر زمان اجرا و هزینه پردازشی سامانه بررسی شود.

برای آشنایی با OpenFHE، مثال‌های موجود در پوشه `src/pke/examples` مخزن این کتابخانه به عنوان مرجع اصلی مورد استفاده قرار می‌گیرند. برای مطالعه مفهوم Hybrid Homomorphic Encryption (HHE) و روش Transciphering نیز از مخزن [Hybrid-HE-Framework](#) و پروژه [TFHE-rs](#) استفاده کنید. این منابع شامل نمونه‌هایی از الگوریتم‌هایی مانند Pasta، Kreyvium، LowMC، Rasta، Masta، FiLIP و Trivium هستند.

همچنین برای مطالعه مبانی کلی و الگوریتم‌ها، از مقاله¹⁰ مورد اشاره استفاده کنید.

مراحل انجام پروژه

- مطالعه مفاهیم FHE، HHE و Transciphering
- مطالعه ساختار و APIهای اصلی کتابخانه OpenFHE
- طراحی معماری Client-Edge-Server
- پیاده‌سازی نسخه Plain سامانه
- پیاده‌سازی ارتباط TCP میان Client، Edge و Server
- پیاده‌سازی تولید کلید، رمزنگاری و رمزگشایی با OpenFHE
- پیاده‌سازی عملیات مورد نیاز با استفاده از Schemeهای مناسب FHE
- پیاده‌سازی روش Transciphering
- پیاده‌سازی محاسبه صورت حساب، میانگین و واریانس روی داده‌های رمز شده
- مقایسه عملکرد نسخه Plain و نسخه مبتنی بر FHE
- بررسی تأثیر Schemeها و پارامترهای مختلف بر کارایی سامانه
- مستندسازی نتایج نهایی پروژه

¹⁰ Abdinasibfar, Hossein, Camille Nuoskala, and Antonis Michalas. "The hhe land: Exploring the landscape of hybrid homomorphic encryption." ACM Computing Surveys 58, no. 12 (2026): 1-35.

توضیحات

در این پروژه، هدف آن است که هشت ابزار زیر در حوزه فازینگ ثابت‌افزار مورد بررسی، تحلیل و مقایسه قرار گیرند:

SHiFT, CO3, EL3XIR, EM-Fuzz, SmuFuzz, TWFuzz, FirmRCA, RT-Fuzzer

شرح پروژه

در گزارش پایانی، ابتدا معرفی اجمالی هر یک از ابزارهای مورد بررسی ارائه شود. در این معرفی، هدف طراحی، دامنه‌ی کاربرد و مسئله‌ی محوری که هر ابزار برای حل آن ارائه شده است، به‌صورت روشن تشریح گردد. در ادامه، تحلیل تفصیلی هر ابزار بر اساس معیارهای فنی و تحلیلی زیر انجام شود.

۱. مشخص گردد که تمرکز اصلی هر ابزار عمدتاً بر بازمیزبانی ثابت‌افزار است یا بر فازینگ. دلایل این انتخاب طراحی با توجه به مسئله‌ای که ابزار در پی حل آن بوده است، تحلیل شود.
۲. ویژگی‌ها و نوآوری‌های منحصربه‌فرد هر ابزار که آن را از سایر رویکردها متمایز می‌کند، با ذکر جزئیات فنی بررسی شود. در این بخش مشخص گردد هر ابزار چه مسئله‌ای را به‌طور خاص هدف قرار داده، چه راهکاری برای آن ارائه کرده و این راهکار چه مزایا و محدودیت‌هایی به همراه داشته است. برای مثال در ابزار Fuzzware برای اولین بار بحث سربار ورودی‌ها مورد بحث گرفت.
۳. معماری کلی هر ابزار و اجزای اصلی به‌کاررفته در آن تشریح گردد. این بخش باید شامل نوع فازر مورد استفاده، شبیه‌ساز یا موتور اجرای به‌کاررفته باشد. در ابزارهایی که از اجزای مکملی نظیر موتور *angr* مشابه استفاده می‌کنند، نقش این اجزا، دلیل استفاده و کاربرد آن‌ها به‌طور جداگانه بررسی شود.
۴. پیش‌نیازهای هر ابزار برای اجرای فازینگ بر روی یک ثابت‌افزار یا معماری سخت‌افزاری خاص بررسی شود. این پیش‌نیازها می‌توانند شامل نیاز به اطلاعات جانبی، پیکربندی‌های خاص، مدل‌های محیطی، یا محدودیت‌های وابسته به معماری باشند.
۵. معماری‌های سخت‌افزاری پشتیبانی‌شده توسط هر ابزار بررسی شوند. این تحلیل باید شامل معماری‌هایی نظیر MIPS, ARM و x86 بوده و دامنه‌ی کاربرد عملی هر ابزار را روشن سازد.
۶. شیوه‌ی تعامل فازر با محیط شبیه‌سازی به‌طور کامل بررسی گردد. این بررسی باید شامل نحوه‌ی تولید و هدایت ورودی‌ها، تزریق آن‌ها به برنامه‌ی هدف و نحوه‌ی دریافت فیدبک (مانند پوشش کد) از محیط اجرا باشد.
۷. رویکرد هر ابزار برای فعال‌سازی وقفه‌ها بررسی گردد. در این بخش، الگوریتم یا منطق مورد استفاده برای شبیه‌سازی یا فعال‌سازی وقفه‌ها تشریح شده و تأثیر آن بر پوشش کد و اثربخشی فازینگ تحلیل شود.
۸. انواع ثبات‌های مورد استفاده در ثابت‌افزار، از جمله ثبات‌های وضعیت دسته‌بندی شده و نحوه‌ی برخورد ابزار با هر یک از این دسته‌ها تشریح گردد.
۹. بررسی شود که کدام ابزارها عملیات DMA را خارج از محدوده‌ی پشتیبانی خود در نظر می‌گیرند و کدام ابزارها اقدام به پیاده‌سازی یا مدل‌سازی آن می‌کنند. همچنین، دلایل فنی رایج برای خارج از محدوده در نظر گرفتن DMA در اغلب ابزارها بررسی شده و در صورت پشتیبانی، نحوه‌ی مدیریت و مدل‌سازی DMA تشریح گردد.

۱۰. تکنیک‌های کلی مورد استفاده در هر ابزار برای تحلیل و فازینگ ثابت‌افزار بررسی شوند. این تکنیک‌ها می‌توانند شامل رویکردهای مختلف بازمیزبانی، شبیه‌سازی یا مدل‌سازی دستگاه‌های جانبی، اجرای مبتنی بر نمای لحظه‌ای^{۱۱} و اجرای نمادین^{۱۲} باشند. برای هر تکنیک، توضیحی خلاصه و دقیق ارائه گردد.
۱۱. تکنیک‌های خاص به‌کاررفته در فرایند اجرای هر ابزار مشخص شوند. این تکنیک‌ها می‌توانند شامل استنتاج الگوهای دسترسی^{۱۳} به ثبات‌ها، ثبت و بازپخش اجرا^{۱۴}، شبیه‌سازی سطح بالا^{۱۵} (HLE) یا سایر روش‌های اختصاصی باشند.
۱۲. نحوه‌ی تولید ورودی‌های اولیه در هر ابزار بررسی شود. مشخص گردد که آیا این ورودی‌ها به‌صورت تصادفی، مبتنی بر تحلیل ایستا، استخراج از ثابت‌افزار، یا بر اساس دانش دامنه تولید می‌شوند.
۱۳. نحوه‌ی تعریف، محاسبه و جمع‌آوری پوشش کد در هر ابزار بررسی شود و مشخص گردد پوشش کد چگونه به‌عنوان معیار هدایت و ارزیابی فرایند فازینگ مورد استفاده قرار می‌گیرد.
۱۴. دو چالش Path Explosion و Path Elimination تعریف شوند. سپس بررسی گردد که هر یک از ابزارها با کدامیک از این چالش‌ها مواجه بوده‌اند و چه راهکارهایی برای کاهش یا مدیریت این مشکلات ارائه داده‌اند.
۱۵. چالش‌ها و محدودیت‌هایی که هر ابزار در مرحله‌ی طراحی، پیاده‌سازی یا ارزیابی با آن‌ها مواجه بوده است، مورد بررسی و تحلیل قرار گیرد.
۱۶. تمامی Trade-off‌های مطرح‌شده و به‌کاررفته در مقالات مربوط به هر ابزار به‌صورت کامل استخراج، تحلیل و گزارش شوند. این تحلیل باید نشان دهد هر ابزار در ازای چه مزایایی، چه محدودیت‌ها یا هزینه‌هایی را پذیرفته است.
۱۷. بر اساس مقالات موجود، چالش‌های فازینگ مستقیم بر روی سخت‌افزار واقعی و رویکرد Hardware-in-the-Loop مورد بررسی قرار گیرند. این بخش باید محدودیت‌ها، هزینه‌ها، مشکلات مقیاس‌پذیری و چالش‌های عملی این رویکردها را تشریح کند.
۱۸. بررسی شود که هر ابزار فرایند کشف و تشخیص خطا و کرش را به چه صورت انجام می‌دهد. در این بخش باید منطق تشخیص خطا، منابع فیدبک مورد استفاده، و ابزارها یا سازوکارهای به‌کاررفته برای شناسایی کرش‌ها تشریح گردد.
۱۹. نحوه‌ی مدیریت و تریاز کرش‌ها در هر ابزار تحلیل گردد. این تحلیل باید شامل روش‌های شناسایی و حذف کرش‌های تکراری^{۱۶}، نحوه‌ی گروه‌بندی خطاها و میزان پشتیبانی ابزار از تحلیل ریشه‌ی خطا باشد.
۲۰. مواردی که منجر به ایجاد نتایج مثبت کاذب^{۱۷} در هر ابزار شده‌اند شناسایی شده و دلایل فنی بروز آن‌ها بررسی گردد.
۲۱. نوع و تعداد باگ‌ها و آسیب‌پذیری‌هایی که هر ابزار موفق به کشف آن‌ها شده است بررسی گردد. این بررسی باید شامل دسته‌بندی باگ‌ها، میزان تکرارپذیری، و سطح تأثیر آن‌ها باشد.
۲۲. روش‌های ارزیابی انجام‌شده توسط هر ابزار و معیارهای کارایی مورد استفاده در ارزیابی‌ها بررسی شوند.
۲۳. بررسی شود که هر ابزار ارزیابی خود را بر روی چه مجموعه‌ای از ثابت‌افزارها انجام داده است. این بررسی باید شامل واقعی یا مصنوعی بودن ثابت‌افزارها، تعداد نمونه‌ها، معماری‌های هدف و میزان تنوع مجموعه‌ی ارزیابی باشد.
۲۴. میزان اتوماسیون و تلاش مهندسی مورد نیاز برای استفاده از هر ابزار بررسی گردد. در این راستا، میزان دخالت دستی مورد نیاز برای راه‌اندازی، پیکربندی‌های لازم، پیچیدگی فرایند استفاده و وابستگی به دسترسی به کد منبع ثابت‌افزار تحلیل شود تا میزان خودکار بودن هر ابزار مشخص گردد.
۲۵. دسترس‌پذیری هر ابزار به‌عنوان یکی از معیارهای مقایسه بررسی گردد. در این بخش، وضعیت متن‌باز یا بسته بودن ابزار، میزان دسترسی به کد منبع و در صورت وجود، در دسترس بودن دیتاست‌ها یا سناریوهای ارزیابی استفاده‌شده در مقالات بررسی شود.

¹¹ Snapshot

¹² Symbolic Execution

¹³ Access pattern inference

¹⁴ Record and replay

¹⁵ High-Level Emulation

¹⁶ Deduplication

¹⁷ False Positives